
Artificial Intelligence

BS (CS) Spring 2026

Lab 13 Manual



Learning Objectives:

1. Supervised Learning

Lab Manual

Supervised Learning:

Supervised learning is a type of machine learning where a model is trained using labeled data. In labeled data, each input is paired with a known output, allowing the model to learn patterns and relationships between them. This structured approach enables the model to make accurate predictions for new, unseen data.

As an example consider supervised learning as similar as guiding a student. The labeled data acts like a tutor, providing correct answers so the model (the student) can learn over time. Through repeated exposure to this labeled data, the model gradually becomes better at predicting the correct outcome on its own.

The main goal of supervised learning is to generalize from the labeled data so that it can make reliable predictions or decisions for unknown data in the future.

How Does Supervised Learning Work?

The process of supervised learning involves several key steps to train a model effectively:

- **Data Preparation** – The first step is preparing the data by cleaning and pre-processing it. This includes removing any irrelevant information, handling missing values, and organizing the data into labeled pairs, where each input has a corresponding output.
- **Training the Model** – The model is then trained using this labeled data. During training, the model analyzes the patterns in the input data and learns to map these inputs to their correct outputs. This involves adjusting the model's parameters to improve accuracy.
- **Model Evaluation** – Once trained, the model is tested on unseen data to evaluate its performance. This helps determine how well the model can generalize beyond the training data.
- **Making Predictions** – Finally, after the model has been trained and tested, it can be used to make predictions on new data. The better it learned during training, the more accurate its predictions will be.

Supervised Learning Algorithms

Supervised learning includes a variety of algorithms, each suited for different types of tasks. Here are the common supervised learning algorithms covered in this lab manual:

- **Decision Tree** – A tree-structured model that makes decisions by splitting data based on feature values. Highly interpretable and used for both classification and regression.
- **Linear Regression** – A simple algorithm that predicts continuous values by finding a linear relationship between input features and the target variable.
- **Logistic Regression** – Used for classification tasks. It predicts binary or multi-class outcomes using the sigmoid function to output probabilities.
- **K-Nearest Neighbors (KNN)** – Classifies or predicts based on the majority label or average value among the k closest data points.
- **Support Vector Machine (SVM)** – Creates an optimal hyperplane to separate classes. Effective for high-dimensional data.
- **Naive Bayes** – A probabilistic classifier based on Bayes' theorem. Very fast and effective for text classification.

- **Random Forest** – An ensemble of decision trees that reduces overfitting and improves accuracy through bagging and feature randomness.
- **Gradient Boosting** – An ensemble technique that builds models sequentially, each correcting errors of the previous. State-of-the-art accuracy.

Types of Supervised Learning

Supervised learning generally falls into two main types, each designed to handle different kinds of problems:

- **Classification** – The model predicts discrete categories or classes. For example, an email can be classified as "spam" or "not spam," or an image as "cat" or "dog." Classification is useful for tasks where outcomes fall into specific groups.
- **Regression** – The model predicts continuous values. For instance, predicting the price of a house based on features like location, size, and age. Regression is used when the output is a numeric range rather than distinct categories.

Algorithm Comparison Table

Algorithm	Type	Purpose	Method	Strengths	Use Cases
Decision Tree	Both	Model decisions & outcomes	Tree-like splits	Interpretable, no scaling	Classification & Regression
Linear Regression	Regression	Predict continuous values	Least squares linear fit	Simple, fast	Price prediction, forecasting
Logistic Regression	Classification	Predict binary/multi-class	Sigmoid function	Probabilistic output	Spam, disease detection
KNN	Both	Predict by proximity	k nearest neighbors	No training phase	Recommendation, recognition
SVM	Both	Optimal class separation	Maximizing margin hyperplane	High-dimensional data	Image classification, NLP
Naive Bayes	Classification	Probabilistic classification	Bayes' theorem	Very fast, text tasks	Spam filtering, sentiment
Random Forest	Both	Ensemble of trees	Bagging + feature randomness	Robust, high accuracy	Finance, healthcare
Gradient Boosting	Both	Sequential error correction	Iterative residual fitting	State-of-the-art accuracy	Competitions, ranking

Basic Structure

The main steps performed on any dataset in supervised learning generally include the following phases:

Data Collection

Gather relevant, representative, and sufficient data for the problem. The quality of data directly determines the quality of the learned model.

Load and Transform Dataset

Load the dataset from files (CSV, Excel, database) into a suitable data structure such as a Pandas DataFrame. Apply initial transformations such as parsing dates, renaming columns, or merging tables.

Data Exploration and Analysis

Understanding of the relevant dataset through exploratory analysis.

Exploratory Data Analysis (EDA)

Understand the dataset by summarizing its main characteristics, often using visual methods such as histograms, box plots, scatter plots, and correlation heatmaps.

Statistical Analysis

Calculate descriptive statistics (mean, median, mode, standard deviation) to understand the data distribution.

- **Statistical Summary:** Use `.describe()` and `.info()` to get an overview of all features.
- **Skewness and Kurtosis Check:** Skewness and kurtosis describe the asymmetry and tail behavior of data, helping to detect outliers and check whether data meets assumptions for statistical analysis and machine learning models.
- **Distribution of Engagement Features:** Distribution of engagement features helps us understand user behavior patterns, detect anomalies, and ensure the data is suitable for accurate modeling and analysis.
- **Correlation Analysis:** Measures the strength and direction of relationships between variables. We want high correlation between independent features and the dependent variable (useful predictors) and low correlation between independent features themselves (to avoid multicollinearity).
- **Outlier Detection:** The process of identifying data points significantly different from most of the data, as they can distort analysis and reduce model accuracy.
- **Z-Score:** Measures how many standard deviations a data point is from the mean, used to detect outliers and standardize data.
- **Zero Ratio Analysis:** Measures the proportion of zero values in a dataset, used to identify sparse features that may carry little information.
- **Group Analysis:** Examines data by dividing it into meaningful categories to compare patterns and differences across groups.
- **Hypothesis Testing:** A statistical method to determine whether there is enough evidence in data to support or reject a specific claim.
- **Variance Check:** Measures how much data values spread around the mean to understand data variability.
- **Feature Insight:** Understanding how each feature behaves and contributes to the data, used to guide feature selection and preprocessing.

Data Cleaning

- **Handling Missing Values:** Impute with mean/median/mode, or drop rows/columns with excessive missing data.

- **Handling Outliers:** Remove outliers (IQR method), Z-score method (remove $|Z| > 3$), Capping (Winsorization), Transformation (log/sqrt), or Imputation.
- **Correcting Data Types:** Ensure numeric columns are not stored as strings and vice versa.

Data Preprocessing

- **Normalization/Standardization:** MinMax Scaler scales numeric values into a fixed range (0 to 1). StandardScaler standardizes features to have mean=0 and std=1.
- **Encoding Categorical Variables:** Label Encoder converts categories into numbers. One-Hot Encoder creates separate binary columns for each category.
- **Feature Selection/Extraction:** Select the most relevant features using techniques like correlation analysis, recursive feature elimination, or PCA.
- **Data Splitting:** Split data into training, validation, and test sets (commonly 70/15/15 or 80/20 split).

Model Selection, Training & Evaluation

- **Choosing the Right Model:** Based on the problem type (classification vs regression), dataset size, and interpretability requirements.
- **Model Training:** Fit the model to the training data. Tune hyperparameters using cross-validation or grid search.
- **Model Evaluation:** Evaluate using appropriate metrics — accuracy, precision, recall, F1-score for classification; RMSE, MAE, R^2 for regression.
- **Model Deployment:** Integrate the trained model into applications and monitor its performance over time.

Decision Tree Model

A Decision Tree is one of the most intuitive and widely-used supervised learning algorithms. It models decisions by constructing a tree-like graph of choices, consequences, and outcomes. Each path from root to leaf represents a classification or regression rule.

Key Components of a Decision Tree

- **Root Node:** The topmost node representing the entire dataset. It is the starting point of the decision-making process and corresponds to the best predictor feature.
- **Internal Nodes (Decision Nodes):** Nodes that represent a feature/attribute and a split condition. Each internal node asks a question about a feature and branches out based on the answer.
- **Branches (Edges):** The arrows or lines connecting nodes. Each branch represents the outcome of a test and leads to either another internal node or a leaf node.
- **Leaf Nodes (Terminal Nodes):** The final nodes that do not split further. They represent the output label (classification) or the predicted value (regression).

How Decision Trees Work – Step by Step

1. **Select the Best Feature to Split:** The algorithm evaluates all features using a splitting criterion (Gini Impurity, Information Gain, or Variance Reduction) and chooses the feature that best separates the data into pure subsets.
2. **Split the Dataset:** The dataset is divided into subsets based on the selected feature's values. For categorical features, each value becomes a branch. For continuous features, a threshold is selected.

3. **Repeat Recursively:** Steps 1 and 2 are repeated for each resulting subset until a stopping condition is met (all samples belong to one class, maximum depth reached, or minimum samples per node reached).
4. **Assign Output at Leaf Nodes:** For classification, the majority class label is assigned. For regression, the mean value of the target variable in that node is assigned.

Splitting Criteria

- **1. Gini Impurity (used in CART):** Measures the probability of a randomly selected element being incorrectly classified. A Gini value of 0 means the node is perfectly pure. Formula: $Gini = 1 - \sum (p_i)^2$, where p_i is the proportion of samples in class i .
- **2. Information Gain / Entropy (used in ID3, C4.5):** Entropy measures the disorder in the dataset. Information Gain is the reduction in entropy achieved after splitting on a feature. Higher Information Gain means a better split. Formula: $Entropy = -\sum (p_i \times \log_2(p_i))$.
- **3. Variance Reduction (for Regression Trees):** Used when the target variable is continuous. The feature that minimizes the weighted variance of the child nodes is selected as the best split.

Types of Decision Trees

- **Classification Tree:** Used when the target variable is categorical (e.g., spam/not spam). The leaf node outputs a class label based on majority voting among training samples in that leaf.
- **Regression Tree:** Used when the target variable is continuous (e.g., house price). The leaf node outputs the average value of the training samples it contains.

Overfitting and Pruning

Decision trees are prone to overfitting, where the model memorizes the training data too precisely and performs poorly on unseen data. Pruning is the technique used to address this problem.

- **Pre-Pruning (Early Stopping):** Stop growing the tree before it perfectly fits the training data. Parameters include `max_depth` (maximum depth), `min_samples_split` (minimum samples to split a node), and `min_samples_leaf` (minimum samples in a leaf node).
- **Post-Pruning (Cost Complexity Pruning):** Grow the full tree first, then remove branches that provide little predictive power. The `ccp_alpha` hyperparameter in scikit-learn controls the aggressiveness of pruning.

Advantages and Disadvantages

- **Advantages:** Easy to understand and visualize; requires little data preprocessing (no normalization needed); handles both numerical and categorical data; identifies the most important features naturally; works well with small datasets.
- **Disadvantages:** Prone to overfitting on noisy data; unstable (small changes in data can produce very different trees); biased toward features with more levels; not suitable for regression with continuous outputs unless pruned carefully.

Python Implementation (scikit-learn)

```
# Step 1: Import Libraries
from sklearn.datasets import load_iris
from sklearn.tree import DecisionTreeClassifier, export_text, plot_tree
from sklearn.model_selection import train_test_split
from sklearn.metrics import accuracy_score, classification_report
import matplotlib.pyplot as plt
```

```

# Step 2: Load and Split Dataset
iris = load_iris()
X, y = iris.data, iris.target
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2,
random_state=42)

# Step 3: Train the Decision Tree
model = DecisionTreeClassifier(criterion='gini', max_depth=4, random_state=42)
model.fit(X_train, y_train)

# Step 4: Evaluate the Model
y_pred = model.predict(X_test)
print('Accuracy:', accuracy_score(y_test, y_pred))
print(classification_report(y_test, y_pred, target_names=iris.target_names))

# Step 5: Visualize the Tree
plt.figure(figsize=(20, 10))
plot_tree(model, feature_names=iris.feature_names,
class_names=iris.target_names, filled=True)
plt.title('Decision Tree Visualization')
plt.show()

```

Key Hyperparameters

- **criterion:** "gini" for Gini Impurity or "entropy" for Information Gain. Default is "gini".
- **max_depth:** Controls the maximum depth of the tree. Limiting this value prevents overfitting. If None, the tree grows until all leaves are pure.
- **min_samples_split:** The minimum number of samples required to split an internal node. Higher values prevent overfitting.
- **min_samples_leaf:** The minimum number of samples that must be present in a leaf node. Larger values smooth the decision boundary.
- **max_features:** Number of features to consider when looking for the best split. Can be an integer, float, "sqrt", or "log2".

Real-World Use Cases

- **Medical Diagnosis:** Classifying patients as having or not having a disease based on symptoms and test results.
- **Credit Risk Assessment:** Banks use decision trees to decide whether to approve or reject loan applications based on income, credit score, and employment history.
- **Customer Segmentation:** E-commerce companies classify customers by purchase behavior and demographics to create targeted marketing campaigns.
- **Fraud Detection:** Identifying fraudulent transactions in financial systems by analyzing transaction patterns.

Support Vector Machine (SVM)

Support Vector Machine is a powerful supervised learning algorithm used for both classification and regression. The core idea is to find the optimal hyperplane that maximally separates classes in the feature space.

What is SVM?

A Support Vector Machine (SVM) finds the decision boundary (hyperplane) that best separates two classes with the maximum margin. The margin is the distance between the hyperplane and the nearest data points from each class — these nearest points are called support vectors. The larger the margin, the better the model generalizes to new data.

For non-linearly separable data, SVM uses the kernel trick to project data into a higher-dimensional space where it becomes linearly separable.

Key Concepts

- **Hyperplane:** A decision boundary that separates classes. In 2D space, it is a line; in 3D, it is a plane; in n-dimensional space, it is a hyperplane.
- **Support Vectors:** The data points closest to the hyperplane. These are the most critical points — the hyperplane is determined entirely by them.
- **Margin:** The distance between the hyperplane and the support vectors. SVM maximizes this margin to improve generalization.
- **Hard Margin:** Requires perfect separation of classes (no misclassifications). Only works when data is linearly separable.
- **Soft Margin:** Allows some misclassifications for better generalization on noisy data. Controlled by hyperparameter C.

The Kernel Trick

When data is not linearly separable, SVM uses the kernel trick to transform data into a higher-dimensional space where a linear separator can be found. Common kernels include:

- **Linear Kernel:** $K(x, y) = x \cdot y$. Suitable for linearly separable data. Fast and efficient for high-dimensional data (e.g., text classification).
- **Polynomial Kernel:** $K(x, y) = (\gamma x \cdot y + r)^d$. Suitable for data with polynomial relationships. Parameter d controls the degree of the polynomial.
- **RBF (Radial Basis Function) / Gaussian Kernel:** $K(x, y) = \exp(-\gamma \|x - y\|^2)$. Most commonly used kernel. Handles complex non-linear boundaries. Works well in most cases.
- **Sigmoid Kernel:** $K(x, y) = \tanh(\gamma x \cdot y + r)$. Similar to neural network activation functions. Less commonly used.

SVM for Regression (SVR)

Support Vector Regression (SVR) uses SVM principles for regression tasks. Instead of maximizing the margin between classes, SVR fits a tube around the regression line (called the ϵ -tube). Data points within the tube are ignored; only points outside contribute to the model.

- **Epsilon (ϵ):** Defines the width of the tube. Larger ϵ means more tolerance for error.
- **C:** Controls the trade-off between tube width and the number of errors allowed.

Advantages and Disadvantages

- **Advantages:** Effective in high-dimensional spaces; memory efficient (only support vectors are stored); versatile through kernel choice; works well for small and medium datasets; robust to overfitting in high-dimensional spaces.
- **Disadvantages:** Slow to train on large datasets ($O(n^2)$ to $O(n^3)$); requires feature scaling; sensitive to hyperparameter choice; not directly interpretable; kernel selection can be challenging.

Python Implementation

```
from sklearn.svm import SVC, SVR
from sklearn.model_selection import train_test_split, GridSearchCV
from sklearn.metrics import accuracy_score, classification_report
from sklearn.preprocessing import StandardScaler
from sklearn.datasets import load_iris

# Load dataset
iris = load_iris()
X, y = iris.data, iris.target

# Feature scaling (MANDATORY for SVM)
scaler = StandardScaler()
X_scaled = scaler.fit_transform(X)

# Split data
X_train, X_test, y_train, y_test = train_test_split(X_scaled, y, test_size=0.2,
random_state=42)

# Train SVM with RBF kernel
svm_model = SVC(kernel='rbf', C=1.0, gamma='scale', probability=True)
svm_model.fit(X_train, y_train)

# Evaluate
y_pred = svm_model.predict(X_test)
print('SVM Accuracy:', accuracy_score(y_test, y_pred))
print(classification_report(y_test, y_pred, target_names=iris.target_names))

# Hyperparameter tuning with GridSearchCV
param_grid = {'C': [0.1, 1, 10, 100], 'gamma': ['scale', 'auto', 0.01, 0.1],
              'kernel': ['rbf', 'linear', 'poly']}
grid_search = GridSearchCV(SVC(), param_grid, cv=5, scoring='accuracy')
grid_search.fit(X_train, y_train)
print('Best Parameters:', grid_search.best_params_)
```

Key Hyperparameters

- **C:** Regularization parameter. Controls trade-off between smooth decision boundary and classifying training points correctly. Higher C = less regularization.
- **kernel:** Kernel function to use: 'linear', 'poly', 'rbf' (default), 'sigmoid'.
- **gamma:** Kernel coefficient for 'rbf', 'poly', 'sigmoid'. 'scale' (default) or 'auto'. Higher gamma = more complex boundary.
- **degree:** Degree of polynomial kernel. Only relevant when kernel='poly'.
- **epsilon (SVR only):** Epsilon in the epsilon-SVR model. Width of the tube around the regression line.

Real-World Use Cases

- **Image Classification:** Handwritten digit recognition, face detection, and object recognition.

- **Text Classification:** Spam detection, sentiment analysis, and document categorization using linear SVM.
- **Bioinformatics:** Cancer classification from gene expression data.
- **Financial Forecasting:** Predicting stock movements and credit risk assessment.

Naive Bayes Classifier

Naive Bayes is a probabilistic classifier based on Bayes' Theorem. It assumes that all features are conditionally independent given the class label. Despite this simplifying assumption, it performs remarkably well in many real-world scenarios.

What is Naive Bayes?

Naive Bayes is a family of simple probabilistic classifiers that apply Bayes' theorem with the "naive" assumption of conditional independence between every pair of features given the class variable. Despite this simplification, Naive Bayes classifiers work well in practice and are especially popular for text classification tasks.

Bayes' Theorem

The fundamental equation underlying Naive Bayes:

$$P(\text{Class} \mid \text{Features}) = P(\text{Features} \mid \text{Class}) \times P(\text{Class}) / P(\text{Features})$$

- **P(Class | Features):** Posterior probability — the probability of a class given the observed features.
- **P(Features | Class):** Likelihood — the probability of observing the features given the class.
- **P(Class):** Prior probability — the base probability of the class before seeing any features.
- **P(Features):** Evidence — the probability of observing the features (normalizing constant).

The classifier picks the class with the highest posterior probability.

Types of Naive Bayes Classifiers

- **Gaussian Naive Bayes:** Assumes that continuous features follow a Gaussian (normal) distribution. Used for numerical features like measurements in the Iris dataset.
- **Multinomial Naive Bayes:** Suitable for discrete count data such as word counts in text classification. Commonly used in spam detection and document classification.
- **Bernoulli Naive Bayes:** Similar to Multinomial NB but for binary features (0/1). Suitable for text data where features represent word presence/absence.
- **Complement Naive Bayes:** A variant of Multinomial NB that uses statistics from all classes except one to improve performance on imbalanced datasets.

The Naive Independence Assumption

The key "naive" assumption is that all features are independent of each other given the class label:

$$P(x_1, x_2, \dots, x_n \mid C) = P(x_1 \mid C) \times P(x_2 \mid C) \times \dots \times P(x_n \mid C)$$

This assumption rarely holds perfectly in real-world data, but it simplifies the computation enormously and often yields surprisingly good results.

Laplace Smoothing

A problem with Naive Bayes is that if a feature value never appears with a particular class in the training data, the probability becomes zero, making the entire prediction zero. Laplace smoothing (add-one smoothing) adds a small constant (typically 1) to all feature counts to avoid zero probabilities:

$$P(x_i | C) = (\text{count}(x_i, C) + \alpha) / (\text{count}(C) + \alpha \times |V|)$$

Where α is the smoothing parameter (set via `var_smoothing` or `alpha` in `sklearn`) and $|V|$ is the vocabulary size.

Advantages and Disadvantages

- **Advantages:** Extremely fast training and prediction; works well with small datasets; handles high-dimensional data efficiently; performs well even with the independence assumption violated; easy to implement and interpret; good for real-time prediction.
- **Disadvantages:** Strong independence assumption often violated in practice; cannot learn relationships between features; can be outperformed by more complex models; poor probability estimates (though class predictions may still be accurate).

Python Implementation

```
from sklearn.naive_bayes import GaussianNB, MultinomialNB, BernoulliNB
from sklearn.model_selection import train_test_split
from sklearn.metrics import accuracy_score, classification_report
from sklearn.datasets import load_iris
from sklearn.preprocessing import StandardScaler

# Gaussian NB for continuous features
iris = load_iris()
X, y = iris.data, iris.target
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2,
                                                    random_state=42)

# Train Gaussian Naive Bayes
gnb_model = GaussianNB()
gnb_model.fit(X_train, y_train)
y_pred = gnb_model.predict(X_test)
print('Gaussian NB Accuracy:', accuracy_score(y_test, y_pred))
print(classification_report(y_test, y_pred, target_names=iris.target_names))

# For text classification using Multinomial NB
from sklearn.feature_extraction.text import CountVectorizer
from sklearn.pipeline import Pipeline

text_pipeline = Pipeline([
    ('vectorizer', CountVectorizer()),
    ('classifier', MultinomialNB(alpha=1.0)) # alpha is Laplace smoothing
])
```

Key Hyperparameters

- **var_smoothing (GaussianNB):** Portion of the largest variance added to variances for stability. Default 1e-9.

- **alpha (MultinomialNB, BernoulliNB):** Laplace smoothing parameter. Default 1.0 (add-one smoothing). Set to 0 to disable smoothing.
- **binarize (BernoulliNB):** Threshold for binarizing features. Features above this value become 1, otherwise 0.
- **fit_prior:** Whether to learn class prior probabilities. Default True. Set to False to use uniform priors.

Real-World Use Cases

- **Email Spam Filtering:** Classifying emails as spam or legitimate based on word occurrences.
- **Sentiment Analysis:** Determining positive or negative sentiment in product reviews or social media.
- **Document Classification:** Categorizing news articles, academic papers, or legal documents by topic.
- **Medical Diagnosis:** Classifying diseases based on symptoms, especially when fast real-time decisions are needed.
- **Recommendation Systems:** Predicting user preferences based on past behavior.

Random Forest

Random Forest is an ensemble learning algorithm that constructs multiple decision trees during training and combines their predictions. By leveraging the wisdom of the crowd, it corrects the tendency of individual decision trees to overfit.

What is Random Forest?

Random Forest is an ensemble method that creates a "forest" of decision trees, each trained on a different random subset of data and features. For classification, it outputs the majority vote across all trees; for regression, it outputs the average prediction. The combination of many diverse trees results in a model that is more robust and accurate than any single tree.

How Random Forest Works

5. **Bootstrap Sampling (Bagging):** From the original training dataset of N samples, create B bootstrap samples (randomly selected with replacement, each of size N). Each decision tree is trained on a different bootstrap sample.
6. **Feature Randomness:** At each split in each tree, only a random subset of features (typically $\sqrt{p}$ for classification, $p/3$ for regression, where p is total features) is considered as split candidates. This decorrelates the trees.
7. **Build Each Tree:** Each decision tree is grown to its full depth (no pruning). The combination of many deep trees with random features prevents overfitting.
8. **Aggregate Predictions:** For classification: majority voting (the class predicted by the most trees wins). For regression: average of all tree outputs.

Out-of-Bag (OOB) Error

Since each tree is trained on a bootstrap sample, about 37% of the original data is left out ("out-of-bag"). These OOB samples can be used to evaluate the model without a separate validation set:

- **OOB Error:** Each training sample is predicted by the trees that were not trained on it. The OOB error provides an unbiased estimate of the model's generalization error.
- Enable OOB evaluation by setting `oob_score=True` in `RandomForestClassifier`.

Feature Importance

Random Forest naturally provides feature importance scores, indicating how much each feature contributes to the model's predictions:

- **Mean Decrease in Impurity (MDI):** The default. Measures how much each feature decreases impurity (Gini/entropy) across all trees.
- **Mean Decrease in Accuracy (MDA):** Permutation-based. Measures the drop in accuracy when a feature's values are randomly shuffled.

Advantages and Disadvantages

- **Advantages:** Highly accurate; robust to overfitting; handles missing values well; provides feature importance; works for both classification and regression; parallelizable (trees are independent); handles imbalanced datasets.
- **Disadvantages:** Less interpretable than a single decision tree; slower prediction for large forests; high memory usage; may not perform well on very sparse data.

Python Implementation

```
from sklearn.ensemble import RandomForestClassifier, RandomForestRegressor
from sklearn.model_selection import train_test_split, cross_val_score
from sklearn.metrics import accuracy_score, classification_report
from sklearn.datasets import load_iris
import pandas as pd
import matplotlib.pyplot as plt

# Load dataset
iris = load_iris()
X, y = iris.data, iris.target
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2,
random_state=42)

# Train Random Forest
rf_model = RandomForestClassifier(
    n_estimators=100,      # number of trees
    max_depth=5,          # max depth of each tree
    max_features='sqrt',  # features per split
    oob_score=True,       # out-of-bag error
    random_state=42,
    n_jobs=-1             # use all CPU cores
)
rf_model.fit(X_train, y_train)

# Evaluate
y_pred = rf_model.predict(X_test)
print('RF Accuracy:', accuracy_score(y_test, y_pred))
print('OOB Score:', rf_model.oob_score_)
print(classification_report(y_test, y_pred, target_names=iris.target_names))

# Feature Importance
importances = pd.Series(rf_model.feature_importances_, index=iris.feature_names)
importances.sort_values().plot(kind='barh', title='Feature Importances')
```

```
plt.tight_layout()
plt.show()
```

Key Hyperparameters

- **n_estimators:** Number of trees in the forest. More trees generally means better performance (up to a point). Default 100.
- **max_depth:** Maximum depth of each tree. None means unlimited (trees grow to full depth). Limiting reduces overfitting.
- **max_features:** Number of features considered at each split. 'sqrt' for classification, 'log2', or an integer.
- **min_samples_split:** Minimum samples required to split an internal node. Larger values prevent overfitting.
- **min_samples_leaf:** Minimum samples required at a leaf node. Acts as smoothing.
- **bootstrap:** Whether to use bootstrap samples. Default True (use bagging).
- **oob_score:** Whether to use out-of-bag samples to estimate generalization score.
- **class_weight:** 'balanced' to handle class imbalance by adjusting sample weights.
- **n_jobs:** Number of parallel jobs for training and prediction. -1 uses all available cores.

Real-World Use Cases

- **Finance:** Credit risk assessment, fraud detection, and stock price prediction.
- **Healthcare:** Disease prediction, drug discovery, and patient outcome classification.
- **E-commerce:** Product recommendation, customer churn prediction, and inventory forecasting.
- **Remote Sensing:** Land cover classification from satellite imagery.

Gradient Boosting

Gradient Boosting is a powerful ensemble technique that builds models sequentially. Each new model focuses on correcting the errors made by the previous model. It is one of the most powerful and widely used algorithms in machine learning competitions.

What is Gradient Boosting?

Gradient Boosting builds an ensemble of weak learners (typically shallow decision trees) sequentially. Each tree is trained to minimize the residual errors of the previous ensemble. The final prediction is the sum of all individual tree predictions, scaled by a learning rate. Unlike Random Forest (which uses bagging), Gradient Boosting uses boosting — trees are added sequentially, not in parallel.

How Gradient Boosting Works – Step by Step

9. **Initialize:** Start with an initial prediction (e.g., the mean of target values for regression, or log-odds for classification).
10. **Calculate Residuals:** Compute the residuals (errors) — the difference between actual values and the current predictions.
11. **Train a Weak Learner:** Fit a shallow decision tree (weak learner) to the residuals. This tree learns to predict the errors of the previous model.
12. **Update Predictions:** Add the new tree's predictions (scaled by the learning rate η) to the running total: $F(x) = F(x) + \eta \times h(x)$.
13. **Repeat:** Steps 2–4 are repeated for the specified number of estimators ($n_estimators$). Each iteration further reduces the residual errors.

The Gradient Descent Connection

Gradient Boosting is framed as gradient descent in function space. Instead of updating model parameters, it updates the entire model (function) at each step to move in the direction that minimizes the loss function. The "gradient" refers to the negative gradient of the loss function — the direction of steepest decrease in error.

Popular Gradient Boosting Libraries

- **scikit-learn GradientBoostingClassifier:** Standard implementation in scikit-learn. Good for learning but slower than specialized libraries.
- **XGBoost (eXtreme Gradient Boosting):** Highly optimized, regularized version. Uses second-order gradients, parallel processing, and built-in regularization.
- **LightGBM (Light Gradient Boosting Machine):** Microsoft's implementation. Very fast using histogram-based algorithm and leaf-wise tree growth. Handles large datasets efficiently.
- **CatBoost:** Yandex's implementation. Handles categorical features natively without preprocessing. Resistant to overfitting.

Advantages and Disadvantages

- **Advantages:** State-of-the-art accuracy on many tasks; handles mixed data types; robust to outliers (with appropriate loss functions); built-in feature importance; flexible loss functions.
- **Disadvantages:** Sequential training — cannot be parallelized like Random Forest; many hyperparameters to tune; prone to overfitting if learning rate is too high; slower training than Random Forest.

Python Implementation

```
from sklearn.ensemble import GradientBoostingClassifier
from sklearn.model_selection import train_test_split
from sklearn.metrics import accuracy_score, classification_report
from sklearn.datasets import load_iris
import matplotlib.pyplot as plt

# Load dataset
iris = load_iris()
X, y = iris.data, iris.target
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2,
                                                    random_state=42)

# Train Gradient Boosting
gb_model = GradientBoostingClassifier(
    n_estimators=100,      # number of boosting stages
    learning_rate=0.1,     # shrinkage factor
    max_depth=3,           # depth of each weak tree
    subsample=0.8,         # fraction of samples per tree
    random_state=42
)
gb_model.fit(X_train, y_train)

# Evaluate
y_pred = gb_model.predict(X_test)
```



```
print('GB Accuracy:', accuracy_score(y_test, y_pred))
print(classification_report(y_test, y_pred, target_names=iris.target_names))

# Using XGBoost (install: pip install xgboost)
import xgboost as xgb
xgb_model = xgb.XGBClassifier(n_estimators=100, learning_rate=0.1, max_depth=3,
                             use_label_encoder=False, eval_metric='mlogloss')
xgb_model.fit(X_train, y_train)
print('XGBoost Accuracy:', accuracy_score(y_test, xgb_model.predict(X_test)))
```

Key Hyperparameters

- **n_estimators:** Number of boosting stages. More stages = more complex model. Use early stopping to find the optimal value.
- **learning_rate (η):** Shrinks each tree's contribution. Lower learning rate = more trees needed = better generalization. Typical range: 0.01–0.3.
- **max_depth:** Depth of individual trees. Shallow trees (2–5) are preferred as weak learners.
- **subsample:** Fraction of training samples used to fit each tree. Introduces randomness and reduces overfitting.
- **min_samples_split / min_samples_leaf:** Control tree growth to prevent overfitting.
- **loss:** Loss function to optimize — 'log_loss' for classification, 'squared_error' or 'absolute_error' for regression.

Early Stopping

Use early stopping to prevent overfitting. Monitor validation loss and stop training when it stops improving:

```
# Early stopping with XGBoost
from sklearn.model_selection import train_test_split
X_tr, X_val, y_tr, y_val = train_test_split(X_train, y_train, test_size=0.2)
xgb_model.fit(X_tr, y_tr,
              eval_set=[(X_val, y_val)],
              early_stopping_rounds=10,
              verbose=False)
```

Real-World Use Cases

- **Kaggle Competitions:** Gradient Boosting (XGBoost, LightGBM) dominates tabular data competitions.
- **Financial Risk:** Credit scoring, fraud detection, and loan default prediction.
- **Search Ranking:** Learning to rank search results by relevance.
- **Healthcare:** Disease diagnosis, drug response prediction, and patient risk stratification.

Model Comparison Summary

The table below provides a comprehensive comparison of all supervised learning models covered in this lab manual to help you choose the right algorithm for your problem:

Model	Task Type	Interpretability	Scaling Required	Training Speed	Strengths	Weaknesses
Decision Tree	Both	High	No	Fast	Easy to visualize, no scaling	Prone to overfitting
Linear Regression	Regression	High	No*	Very Fast	Simple, interpretable	Linear assumption, sensitive to outliers
Logistic Regression	Classification	High	Yes	Very Fast	Probabilistic output, fast	Linear decision boundary
KNN	Both	High	Yes	No training	No training, simple	Slow prediction, memory-heavy
SVM	Both	Low	Yes	Slow (large N)	High-dimensional, kernel trick	Slow on large data, hyperparameter sensitive
Naive Bayes	Classification	High	No	Very Fast	Very fast, text tasks	Independence assumption
Random Forest	Both	Medium	No	Moderate	Robust, feature importance	Less interpretable, memory
Gradient Boosting	Both	Low	No	Slow	State-of-the-art accuracy	Many hyperparameters, slow training

*Linear Regression does not require scaling for plain OLS, but scaling is needed for regularized variants (Ridge, Lasso).

Choosing the Right Algorithm

- **Small dataset, need interpretability:** Decision Tree, Logistic Regression, or Naive Bayes.
- **Large dataset, tabular data, best accuracy:** Gradient Boosting (XGBoost, LightGBM) or Random Forest.
- **Text classification:** Naive Bayes (fast baseline) or SVM (linear kernel).
- **Image, audio, or NLP tasks:** Neural Networks (CNN, RNN, Transformer).
- **High-dimensional data:** SVM (linear kernel) or Logistic Regression with regularization.
- **No clear choice:** Start with Logistic Regression / Decision Tree as baselines, then try Random Forest and Gradient Boosting.